ADVANCED SQL QUERY SYSTEM

AIM:

To implement subqueries and nested queries in the EcoCart E-Commerce Database to analyze customer purchases, product pricing, and sales performance.

OBJECTIVES:

- To understand and implement single-row subqueries.
- To implement multi-row subqueries and nested queries.
- To identify products whose prices are above the average price.
- To analyze customers with maximum purchases.
- To identify high-value customers and top-selling products.
- To generate advanced business reports for decision-making.

IMPLEMENTATION:

1.Implement Subqueries and Nested Queries:

A subquery is a query written inside another SQL query. It helps solve complex problems by using the result of one query in another query.

The following types of subqueries are implemented:

- Single-row Subquery
- Multi-row Subquery
- Nested Subquery

```
SQL> SELECT product_name, price FROM product WHERE price > (SELECT AVG(price) FROM product);

PRODUCT_NAME                PRICE
--------------------------- -------------
Bamboo Tray                 400.00
Steel Lunch Box             600.00
Eco Cotton Kurta            850.00

SQL> SELECT product_name, price FROM product WHERE price = (SELECT MAX(price) FROM product);

PRODUCT_NAME                PRICE
--------------------------- -------------
Eco Cotton Kurta            850.00

SQL> SELECT product_name, price FROM product WHERE category_id IN (SELECT category_id FROM category);

PRODUCT_NAME                PRICE
--------------------------- -------------
Organic Oats                300.00
Natural Honey               320.00
Aloe Face Cream             250.00
Bamboo Comb                 100.00
Bamboo Tray                 400.00
Steel Lunch Box             600.00
Eco Diary                   140.00
Cotton Tote Bag             180.00
Eco Cotton Kurta            850.00
Organic Seed Pack           110.00

10 rows selected.

SQL> SELECT product_name, price FROM product WHERE price > (SELECT AVG(price) FROM product WHERE category_id IN (SELECT category_id FROM category));

PRODUCT_NAME                PRICE
--------------------------- -------------
Bamboo Tray                 400.00
Steel Lunch Box             600.00
Eco Cotton Kurta            850.00
```

2.Find Products Above Average Price:

This query is used to identify premium products in the EcoCart database.

The following details are retrieved:

- Products priced above the average product price.

```
SQL> SELECT product_id, product_name, price FROM product WHERE price > (SELECT AVG(price) FROM product);

PRODUCT_ID   PRODUCT_NAME                    PRICE
------------ ------------------------- -------------
PRO005       Bamboo Tray                     400.00
PRO006       Steel Lunch Box                 600.00
PRO009       Eco Cotton Kurta                850.00
```

- Product name, category, and price.

```
SQL> SELECT p.product_name, c.category_name, p.price FROM product p JOIN category c ON p.category_id = c.category_id WHERE p.price > (SELECT AVG(price) FROM
 product);

PRODUCT_NAME              CATEGORY_NAME                   PRICE
------------------------- ------------------------- -------------
Bamboo Tray               Bamboo Products                400.00
Steel Lunch Box           Kitchen Items                  600.00
Eco Cotton Kurta          Sustainable Fashion            850.00
```

- Expensive products in each category.

```
SQL> SELECT p.product_name, p.category_id, p.price FROM product p WHERE p.price = (SELECT MAX(p2.price) FROM product p2 WHERE p2.category_id = p.category_id
);

PRODUCT_NAME              CATEGORY_ID          PRICE
------------------------- ------------ -------------
Natural Honey             CAT001              320.00
Aloe Face Cream           CAT002              250.00
Bamboo Tray               CAT003              400.00
Steel Lunch Box           CAT004              600.00
Eco Diary                 CAT005              140.00
Cotton Tote Bag           CAT006              180.00
Eco Cotton Kurta          CAT008              850.00
Organic Seed Pack         CAT009              110.00

8 rows selected.
```

- Products with prices higher than a selected category average.

```
SQL> SELECT p.product_name, c.category_name, p.price FROM product p JOIN category c ON p.category_id = c.category_id WHERE p.price > (SELECT AVG(p2.price) F
ROM product p2 WHERE p2.category_id = p.category_id);

PRODUCT_NAME              CATEGORY_NAME                   PRICE
------------------------- ------------------------- -------------
Natural Honey             Organic Food                   320.00
Bamboo Tray               Bamboo Products                400.00
```

3.Identify Customers with Maximum Purchases:

Customer purchasing behavior is analyzed using SQL queries.

The following details are identified:

- Customer who spent the highest amount.

```
SQL> SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.cus
tomer_id, c.customer_name HAVING SUM(o.total_amount) = (SELECT MAX(total_sales) FROM (SELECT SUM(total_amount) AS total_sales FROM orders GROUP BY customer_
id));

CUSTOMER_ID  CUSTOMER_NAME               TOTAL_SPENT
------------ ------------------------- -------------
CUS006       Pavithra M                      850.00
```

- Customers with the maximum number of order.

```
SQL> SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS total_orders FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.cust
omer_id, c.customer_name HAVING COUNT(o.order_id) = (SELECT MAX(order_count) FROM (SELECT COUNT(*) AS order_count FROM orders GROUP BY customer_id));

CUSTOMER_ID  CUSTOMER_NAME            TOTAL_ORDERS
------------ ------------------------- ------------
CUS010       Lavanya M                            1
CUS004       Keerthana S                          1
CUS008       Nithya R                             1
CUS002       Harini Devi                          1
CUS001       Sanjay Kumar                         1
CUS003       Praveen Raj                          1
CUS007       Ashwin Anand                         1
CUS009       Surya Prakash                        1
CUS006       Pavithra M                           1
CUS005       Dinesh Kumar                         1

10 rows selected.
```

- Customers whose spending is above average.

```
SQL> SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.cus
tomer_id, c.customer_name HAVING SUM(o.total_amount) > (SELECT AVG(customer_total) FROM (SELECT SUM(total_amount) AS customer_total FROM orders GROUP BY cus
tomer_id));

CUSTOMER_ID  CUSTOMER_NAME               TOTAL_SPENT
------------ -------------------------- --------------
CUS010       Lavanya M                         520.00
CUS001       Sanjay Kumar                      520.00
CUS003       Praveen Raj                       600.00
CUS006       Pavithra M                        850.00
```

- Top 5 valuable customers.

```
SQL> SELECT * FROM (SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.customer_id
 GROUP BY c.customer_id, c.customer_name ORDER BY total_spent DESC) WHERE ROWNUM <= 5;

CUSTOMER_ID  CUSTOMER_NAME               TOTAL_SPENT
------------ -------------------------- --------------
CUS006       Pavithra M                        850.00
CUS003       Praveen Raj                       600.00
CUS010       Lavanya M                         520.00
CUS001       Sanjay Kumar                      520.00
CUS007       Ashwin Anand                      420.00
```

4.Generate Complex Business Queries:

Advanced SQL queries are created for business analysis.

Query 1: Best-Selling Product

Identifies products with maximum sales quantity and displays product name, total quantity sold, and total revenue generated.

```
SQL> SELECT p.product_name, SUM(od.quantity) AS total_quantity_sold, SUM(od.quantity * od.price) AS total_revenue FROM product p JOIN order_details od ON p.
product_id = od.product_id GROUP BY p.product_id, p.product_name HAVING SUM(od.quantity) = (SELECT MAX(total_quantity) FROM (SELECT SUM(quantity) AS total_q
uantity FROM order_details GROUP BY product_id));

PRODUCT_NAME             TOTAL_QUANTITY_SOLD  TOTAL_REVENUE
------------------------ -------------------- --------------
Cotton Tote Bag                            3         540.00
Eco Diary                                  3         420.00
```

Query 2: High-Value Customers

Identifies customers whose total purchase amount is greater than the average customer spending.

```
SQL> SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spending FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.
customer_id, c.customer_name HAVING SUM(o.total_amount) > (SELECT AVG(customer_spending) FROM (SELECT SUM(total_amount) AS customer_spending FROM orders GRO
UP BY customer_id));

CUSTOMER_ID  CUSTOMER_NAME            TOTAL_SPENDING
------------ ------------------------ --------------
CUS010       Lavanya M                       520.00
CUS001       Sanjay Kumar                    520.00
CUS003       Praveen Raj                     600.00
CUS006       Pavithra M                      850.00
```

Query 3: Category Performance Analysis

Analyzes category-wise sales performance by identifying the highest revenue-generating category, number of products sold in each category, and average product price by category.

```
SQL> SELECT * FROM (SELECT c.category_name, NVL(SUM(od.quantity * od.price), 0) AS total_revenue FROM category c LEFT JOIN product p ON c.category_id = p.ca
tegory_id LEFT JOIN order_details od ON p.product_id = od.product_id GROUP BY c.category_id, c.category_name ORDER BY total_revenue DESC) WHERE ROWNUM = 1;

CATEGORY_NAME            TOTAL_REVENUE
------------------------ --------------
Sustainable Fashion              850.00
```

```
SQL> SELECT c.category_name, NVL(SUM(od.quantity), 0) AS products_sold FROM category c LEFT JOIN product p ON c.category_id = p.category_id LEFT JOIN order_
details od ON p.product_id = od.product_id GROUP BY c.category_id, c.category_name;

CATEGORY_NAME             PRODUCTS_SOLD
------------------------- -------------
Kitchen Items                         1
Eco Stationery                        3
Personal Care                         1
Cloth Bags                            3
Cleaning Products                     0
Sustainable Fashion                   1
Home Decor                            0
Farming Products                      0
Organic Food                          3
Bamboo Products                       1

10 rows selected.
```

```
SQL> SELECT c.category_name, AVG(p.price) AS average_product_price FROM category c JOIN product p ON c.category_id = p.category_id GROUP BY c.category_id, c
.category_name;

CATEGORY_NAME             AVERAGE_PRODUCT_PRICE
------------------------- ---------------------
Kitchen Items                            600.00
Eco Stationery                           140.00
Personal Care                            250.00
Cloth Bags                               180.00
Sustainable Fashion                      850.00
Farming Products                         110.00
Organic Food                             310.00
Bamboo Products                          250.00

8 rows selected.
```

Query 4: Customer Purchase History

Displays customer name, number of orders, total purchase amount, and most purchased product.

```
SQL> WITH order_totals AS (SELECT customer_id, COUNT(order_id) AS number_of_orders, SUM(total_amount) AS total_purchase_amount FROM orders GROUP BY customer
_id), product_totals AS (SELECT o.customer_id, p.product_name, SUM(od.quantity) AS total_quantity, ROW_NUMBER() OVER (PARTITION BY o.customer_id ORDER BY SU
M(od.quantity) DESC, p.product_name) AS rn FROM orders o JOIN order_details od ON o.order_id = od.order_id JOIN product p ON od.product_id = p.product_id GR
OUP BY o.customer_id, p.product_id, p.product_name) SELECT c.customer_name, NVL(ot.number_of_orders, 0) AS number_of_orders, NVL(ot.total_purchase_amount, 0
) AS total_purchase_amount, pt.product_name AS most_purchased_product FROM customer c LEFT JOIN order_totals ot ON c.customer_id = ot.customer_id LEFT JOIN
product_totals pt ON c.customer_id = pt.customer_id AND pt.rn = 1;

CUSTOMER_NAME             NUMBER_OF_ORDERS TOTAL_PURCHASE_AMOUNT MOST_PURCHASED_PRODUCT
------------------------- ---------------- --------------------- -------------------------
Sanjay Kumar                             1                   520 Natural Honey
Harini Devi                              1                   250 Aloe Face Cream
Praveen Raj                              1                   600 Steel Lunch Box
Keerthana S                              1                   400 Bamboo Tray
Dinesh Kumar                             1                   180 Cotton Tote Bag
Pavithra M                               1                   850 Eco Cotton Kurta
Ashwin Anand                             1                   420 Eco Diary
Nithya R                                 1                   360 Cotton Tote Bag
Lavanya M                                1                   520 Natural Honey
Surya Prakash                            1                   200

10 rows selected.
```

5. ADVANCED SQL REPORTS:

Report 1: Premium Product Report

This report displays product name, category, price, and stock availability condition for products whose prices are above the average product price.

```
SQL> SELECT p.product_name, c.category_name, p.price, CASE WHEN p.stock_quantity > 0 THEN 'Available' ELSE 'Out of Stock' END AS stock_availability FROM pro
duct p JOIN category c ON p.category_id = c.category_id WHERE p.price > (SELECT AVG(price) FROM product);

PRODUCT_NAME              CATEGORY_NAME                PRICE STOCK_AVAILABILITY
------------------------- ------------------------- ------- -------------------
Bamboo Tray               Bamboo Products            400.00 Available
Steel Lunch Box           Kitchen Items              600.00 Available
Eco Cotton Kurta          Sustainable Fashion        850.00 Available
```

Report 2: Customer Value Report

This report displays customer name, total orders, total spending, and customer category.

```
SQL> SELECT c.customer_name, COUNT(o.order_id) AS total_orders, NVL(SUM(o.total_amount), 0) AS total_spending, CASE WHEN NVL(SUM(o.total_amount), 0) >= 5000
0 THEN 'Premium' ELSE 'Regular' END AS customer_category FROM customer c LEFT JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.customer_id, c.custo
mer_name;

CUSTOMER_NAME             TOTAL_ORDERS TOTAL_SPENDING CUSTOMER_CATEGORY
------------------------- ------------ -------------- --------------------
Lavanya M                            1         520.00 Regular
Keerthana S                          1         400.00 Regular
Nithya R                             1         360.00 Regular
Harini Devi                          1         250.00 Regular
Sanjay Kumar                         1         520.00 Regular
Praveen Raj                          1         600.00 Regular
Ashwin Anand                         1         420.00 Regular
Surya Prakash                        1         200.00 Regular
Pavithra M                           1         850.00 Regular
Dinesh Kumar                         1         180.00 Regular

10 rows selected.
```

Report 3: Sales Performance Report

This report displays product-wise sales, category-wise revenue, and top-selling products.

```
SQL> SELECT p.product_name, NVL(SUM(od.quantity), 0) AS quantity_sold, NVL(SUM(od.quantity * od.price), 0) AS total_revenue FROM product p LEFT JOIN order_d
etails od ON p.product_id = od.product_id GROUP BY p.product_id, p.product_name;

PRODUCT_NAME              QUANTITY_SOLD  TOTAL_REVENUE
------------------------- ------------- --------------
Cotton Tote Bag                       3         540.00
Bamboo Tray                           1         400.00
Aloe Face Cream                       1         250.00
Steel Lunch Box                       1         600.00
Eco Cotton Kurta                      1         850.00
Organic Oats                          1         200.00
Natural Honey                         2         640.00
Eco Diary                             3         420.00
Bamboo Comb                           0           0.00
Organic Seed Pack                     0           0.00

10 rows selected.
```

```
SQL> SELECT c.category_name, NVL(SUM(od.quantity * od.price), 0) AS total_revenue FROM category c LEFT JOIN product p ON c.category_id = p.category_id LEFT
JOIN order_details od ON p.product_id = od.product_id GROUP BY c.category_id, c.category_name;

CATEGORY_NAME             TOTAL_REVENUE
------------------------- --------------
Kitchen Items                     600.00
Eco Stationery                    420.00
Personal Care                     250.00
Cloth Bags                        540.00
Cleaning Products                   0.00
Sustainable Fashion               850.00
Home Decor                          0.00
Farming Products                    0.00
Organic Food                      840.00
Bamboo Products                   400.00

10 rows selected.
```

```
SQL> SELECT * FROM (SELECT p.product_name, SUM(od.quantity) AS quantity_sold FROM product p JOIN order_details od ON p.product_id = od.product_id GROUP BY p
.product_id, p.product_name ORDER BY quantity_sold DESC) WHERE ROWNUM <= 5;

PRODUCT_NAME              QUANTITY_SOLD
------------------------- -------------
Cotton Tote Bag                       3
Eco Diary                             3
Natural Honey                         2
Bamboo Tray                           1
Eco Cotton Kurta                      1
```

Report 4: Business Decision Report

This report identifies:

- High-performing products.

```
SQL> SELECT p.product_name, SUM(od.quantity) AS quantity_sold, SUM(od.quantity * od.price) AS total_revenue FROM product p JOIN order_details od ON p.produc
t_id = od.product_id GROUP BY p.product_id, p.product_name HAVING SUM(od.quantity) >= (SELECT AVG(product_quantity) FROM (SELECT SUM(quantity) AS product_qu
antity FROM order_details GROUP BY product_id)) ORDER BY total_revenue DESC;

PRODUCT_NAME              QUANTITY_SOLD  TOTAL_REVENUE
------------------------- ------------- --------------
Natural Honey                         2         640.00
Cotton Tote Bag                       3         540.00
Eco Diary                             3         420.00
```

- Low-performing products.

```
SQL> SELECT p.product_name, NVL(SUM(od.quantity), 0) AS quantity_sold, NVL(SUM(od.quantity * od.price), 0) AS total_revenue FROM product p LEFT JOIN order_d
etails od ON p.product_id = od.product_id GROUP BY p.product_id, p.product_name HAVING NVL(SUM(od.quantity), 0) <= (SELECT AVG(product_quantity) FROM (SELEC
T SUM(quantity) AS product_quantity FROM order_details GROUP BY product_id));

PRODUCT_NAME              QUANTITY_SOLD  TOTAL_REVENUE
------------------------- ------------- --------------
Bamboo Tray                           1         400.00
Aloe Face Cream                       1         250.00
Steel Lunch Box                       1         600.00
Eco Cotton Kurta                      1         850.00
Organic Oats                          1         200.00
Bamboo Comb                           0           0.00
Organic Seed Pack                     0           0.00

7 rows selected.
```

- High-value customers.

```
SQL> SELECT c.customer_name, SUM(o.total_amount) AS total_spending FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.customer_id, c.
customer_name HAVING SUM(o.total_amount) > (SELECT AVG(customer_spending) FROM (SELECT SUM(total_amount) AS customer_spending FROM orders GROUP BY customer_
id)) ORDER BY total_spending DESC;

CUSTOMER_NAME             TOTAL_SPENDING
------------------------- --------------
Pavithra M                        850.00
Praveen Raj                       600.00
Lavanya M                         520.00
Sanjay Kumar                      520.00
```

- Revenue trends.

```
SQL> SELECT TO_CHAR(order_date, 'YYYY-MM') AS sales_month, COUNT(order_id) AS total_orders, SUM(total_amount) AS monthly_revenue FROM orders GROUP BY TO_CHA
R(order_date, 'YYYY-MM') ORDER BY sales_month;

SALES_MONTH     TOTAL_ORDERS MONTHLY_REVENUE
--------------- ------------ ---------------
2026-09                 10            4300
```